

cache operations, defined in *address_space_operations*. For these various operations, most of these functions will then interface with the corresponding underlying block device driver to ultimately operate with the formatted file system on the hardware.

To start with, Pugs laid out the complete framework of his real SFS module, but with minimal functionality; good enough to compile, load and not crash the kernel. He populated only the first of these five structures—*struct file_system_type*, and left all the others empty. Here's the exact code of the structure definitions:

```
#include <linux/fs.h> /* For system calls, structures, ... */
#include <linux/version.h> /* For LINUX_VERSION_CODE & KERNEL_
VERSION */

static struct file_system_type sfs;
static struct super_operations sfs_sops;
static struct inode_operations sfs_iops;
static struct file_operations sfs_fops;
static struct address_space_operations sfs_aops;

static struct file_system_type sfs =
{
    name: "sfs", /* Name of our file system */
    if (LINUX_VERSION_CODE < KERNEL_VERSION(2,6,38))
        get_sb: sfs_get_sb,
    #else
        mount: sfs_mount,
    #endif
    kill_sb: kill_block_super,
    owner: THIS_MODULE
};
```



Note that before Linux kernel version 2.6.38, the *mount* function pointer was referred to as *get_sb*, and it also used to have slightly different parameters—hence the above *#if* for it to be compatible at least across 2.6.3x and possibly with 3.x kernel versions (no guarantee for other versions). Accordingly, the corresponding functions *sfs_get_sb* and *sfs_mount* are also *#ifd*, as follows:

```
#include <linux/kernel.h> /* For printk, ... */

if (LINUX_VERSION_CODE < KERNEL_VERSION(2,6,38))
static int sfs_get_sb(struct file_system_type *fs_type, int flags,
const char *devname, void *data, struct vfsmount *vm)
{
    printk(KERN_INFO "sfs: devname = %s\n", devname);

    /* sfs_fill_super this will be called to fill the super block */
    return get_sb_bdev(fs_type, flags, devname, data, &sfs_fill_
super, vm);
}
```



Figure 1: Formatting the pen drive

```
}
#else
static struct dentry *sfs_mount(struct file_system_type *fs_type,
int flags, const char *devname, void *data)
{
    printk(KERN_INFO "sfs: devname = %s\n", devname);

    /* sfs_fill_super this will be called to fill the super block */
    return mount_bdev(fs_type, flags, devname, data, &sfs_fill_
super);
}
#endif
```

The only difference in the above two functions is that, in the latter, the VFS mount-point-related structure has been removed. The *printk* in there would display the underlying partition's device file, which the user is going to mount (the pen drive's SFS-formatted partition). The *get_sb_bdev()* and *mount_bdev()* functions are generic block-device mount functions for the respective kernel versions, defined in *fs/super.c* and prototyped in *<linux/fs.h>*. Pugs also used them, as most other file system writers do. If you are wondering if all file systems mount a block device the same way—well, most of them do; except the part where the mount operation needs to fill in the VFS' superblock structure (*struct super_block*) as per the superblock of the underlying file system—that, most probably, would be different. But then how do file systems do that? If you observe the above functions carefully, apart from passing all the parameters as-is, there is an additional parameter *sfs_fill_super*, and that is Pugs' custom function to fill the VFS' superblock as per the SFS file system.

Unlike the mount function pointer, the unmount function pointer has been the same (*kill_sb*) for quite some kernel versions; and in unmounting, there is not even the minimal distinction required across different file systems. So, the generic block-device unmount function *kill_block_super()* has been used directly as the function pointer.

In *sfs_fill_super()*, Pugs is ideally supposed to read the superblock from the underlying hardware-space SFS, and accordingly translate and fill that into VFS' superblock to enable